

# CAGE: Chimera Assembly with GPU Execution

**Ralph Noack, Ph.D.**
**President**

**Celeritas Simulation Technology, LLC**

***www.CeleritasSimTech.com***



# Outline

- Introduction: GPUs vs CPUs
- Brief Overview of Suggar++
- CAGE overview
- Discuss Overset Assembly Work
- Single vs Multi GPU Execution
- Testcases
- Summary



# Introduction

- GPUs have the potential to provide significant compute power.
- GPUs have significant differences from CPUs that enable this performance:
  - Thousands of specialized cores
  - High bandwidth to dedicated memory



# GPUs vs CPUs



# Contrasting GPU with CPU

- GPU
  - Thousands of simple cores
  - Single Instruction Multiple Thread (SIMT)
    - Performance stall if threads do not have identical execution
- CPU
  - Have few complex cores
  - Each thread can have independent instructions and data.



# Contrasting GPU with CPU: parallelism

- CPU
  - Multiple threads
  - Multiple compute nodes
- GPU
  - Multiple threads
  - Multiple compute nodes
  - Multiple GPUs in compute node
  - Multiple streams/”batch queues”



# Suggar++

Overview and why not suited for GPU execution



# Suggar++ Overview

- General overset assembly code for a wide range of problems
  - Node and/or cell centered solvers
  - Grids: Structured, Unstructured: tet, mixed element, general polyhedral, Octree
- Hole cutting approaches
  - Direct cut, analytic, octree, hybrid, Xray
- Surface assembly for overlapping surfaces
- Lots of other capabilities



# Suggar++ Parallel Execution

- Threads for shared memory machines
  - Thread loops over grids
  - Thread loops over cells/vertices
- Hybrid MPI parallel execution
  - Use MPI to distribute memory across nodes
  - Use threads within a node



# Suggar++ Not Suited for GPUs

- Complex data structures not designed for GPUs
- Donor search allocates a donor for each candidate donor
  - Processed at end to find best one
- Direct hole cut is too complex for GPUs
  - Lots of conditional statements

# CAGE Overview



# CAGE: Chimera Assembly with GPU Execution

- CAGE is a clean slate development targeted for GPUs
  - New design/approaches
  - No code from Suggar++
  - Reduced capability set
    - Currently no overlapping surfaces
    - …



# CAGE Capabilities

- Node or cell centered
- Grids: Unstructured only
  - Mixed element, general polyhedral
- Clone grid data on GPU
- DSF for overlap minimization
  - Cell volume
  - Distance to wall
- Mark Immersed Boundary locations



# CAGE Hole Cut

- Cut with Cartesian Inverse Map
  - Sized to bounding box of solid surfaces
  - Mark map cells as
    - Outside body, Boundary, Inside body
  - Locations in Boundary cells will use donor search for more accurate cut
    - Map is augmented with cells adjacent to boundary to start donor search
- Mark inner(surrounding holes) and outer fringes

# CAGE Donor search

- Search all locations
  - Keep a single best donor
  - Two Cartesian Inverse Maps to start search
    - Hole cut inverse map with boundary elements
    - Map for whole donor Grid
- Post process
  - Identify needed fringes
  - Remove min fringe from locations needed by active location
  - Gather list of fringes



# CAGE Execution Mode

- Run time select
  - CPU Serial
    - Useful for debug
  - CPU threads (Intel TBB)
    - Useful for debug, testing, no GPU available
  - GPU
    - Single or Multiple
  - Same kernel code
    - #include kernel body
- Compile for NVIDIA or AMD GPU



# Overset Assembly Work



# Overset Assembly Work Patterns

- Within a grid

  For grid A in grids:

  For solver locations in grid A:

  Perform task kernel

  - Example: mark needed fringes in a grid

- Between two grids

  For fringe grid A in grids:

  For donor grid B in grids:

  For solver locations in grid A:

  Perform task kernel

  - Requires data transfer between GPUs
  - Example: hole cutting, donor search, donor weights



# Overset Assembly Work Disparity

- Work for one grid can be very different than another
  - Different size grids
  - Large background can have only a small region/elements that overlap
  - Different grid types will have different work
- Work can change with time due to motion
- Difficult to load balance



# Contrast Single vs Multi GPU Execution

- Preallocate memory on GPU/copy data from CPU at initialization
- Single GPU execution
  - Data remains on GPU for all stages
  - Very low memory transfers
  - Data copied to CPU for writing DCI file
- Multi-GPU execution
  - Grid is assigned to a GPU
  - Must copy data to/from worker GPU
    - Lots of data transfer



# How to Reduce MemCPY Time for Multi-GPU Execution

- Pipeline
  - Decompose work into chunks
  - Overlap memcpy for chunk 2 with work on chunk 1 using different streams
- Duplicate data
  - All GPUs contain all needed data
    - Cloned grids reduce memory requirement
  - Each GPU works on grids assigned to it
  - Iblank/flags must be shared with all GPUs



# Test Cases



# Test Hardware

- Development system
  - AMD Ryzen 9 3950X
    - 16-Core/32-threads, 128GB of memory
  - Dual NVIDIA GeForce RTX 3060 ($350 cost)
    - 12GB of memory, 3584 CUDA Cores
    - GPU0 H2D/D2H Bandwidth : 25 GB/s x16 slot
    - GPU1 H2D/D2H Bandwidth : 6 GB/s x4 slot
    - (not) Peer Transfer Bandwidth : 4.80 GB/s
  - Can swap in AMD Radeon RX 9060 XT
    - 16GB of memory, 2048 Cores

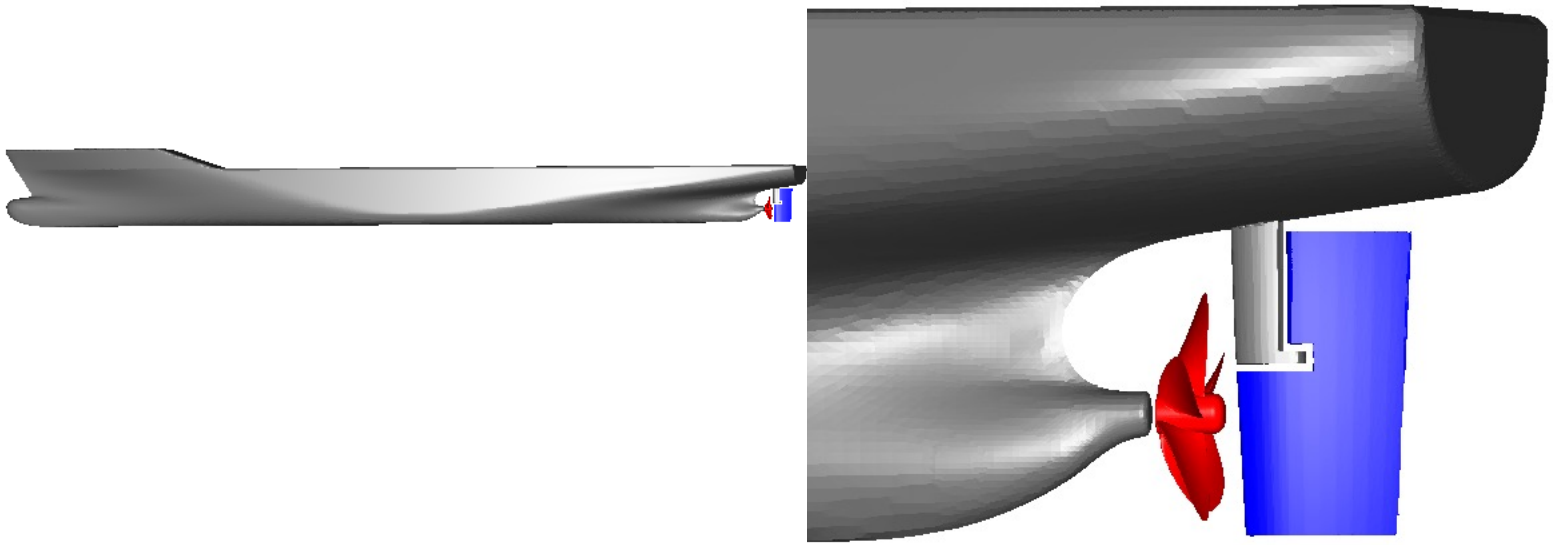


# Test Hardware

- Cloud provider
  - NVIDIA A100-SXM4 6912 CUDA Cores, 80GB of memory
  - H2D/D2H Bandwidth : 26 GB/s
  - Peer Transfer Bandwidth : 278 GB/s
  - $1.59/hour/GPU

# KCS General Polyhedral Grids

- 4.2M nodes, 3.8M cells
  - Background, Hull, Rudder, Propellor grids
  - Octree based/Hex dominant (SnappyHexMesh?)
    - max pts/face 10, faces/cell 21



# KCS General Polyhedral Grids: Wall Clock Comparison

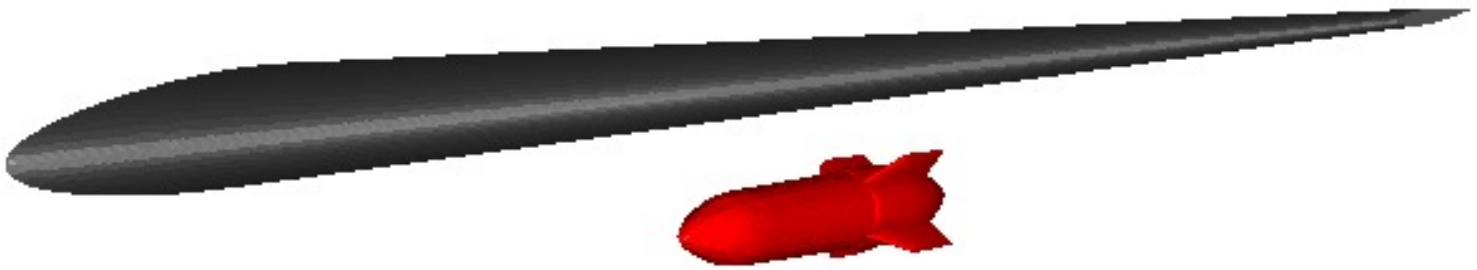
Dual NVIDIA GeForce RTX 3060

| | 1gpu | 2gpu/1gpu | 2gpuD/1gpu | tbb/1gpu | Suggar++/1gpu |
|---|---|---|---|---|---|
| CC | 0.051 | 3.0 | 2.2 | 5.6 | 15.1 |

**CC: Cell Centered**
**1gpu: single gpu (x16) wall clock time in seconds**
**2gpu: two gpus (x16, x4)**
**2gpuD: two gpus with duplication**
**tbb: threaded cpu execution only (16 core/32thr)**
**Suggar++ (16 core/32thr)**

# Wing Store All Tetrahedron Grids

- 8.3M nodes, 49.4M cells
  - Store, Wing grids





# Wing Store All Tetrahedron Grids: Wall Clock Comparison

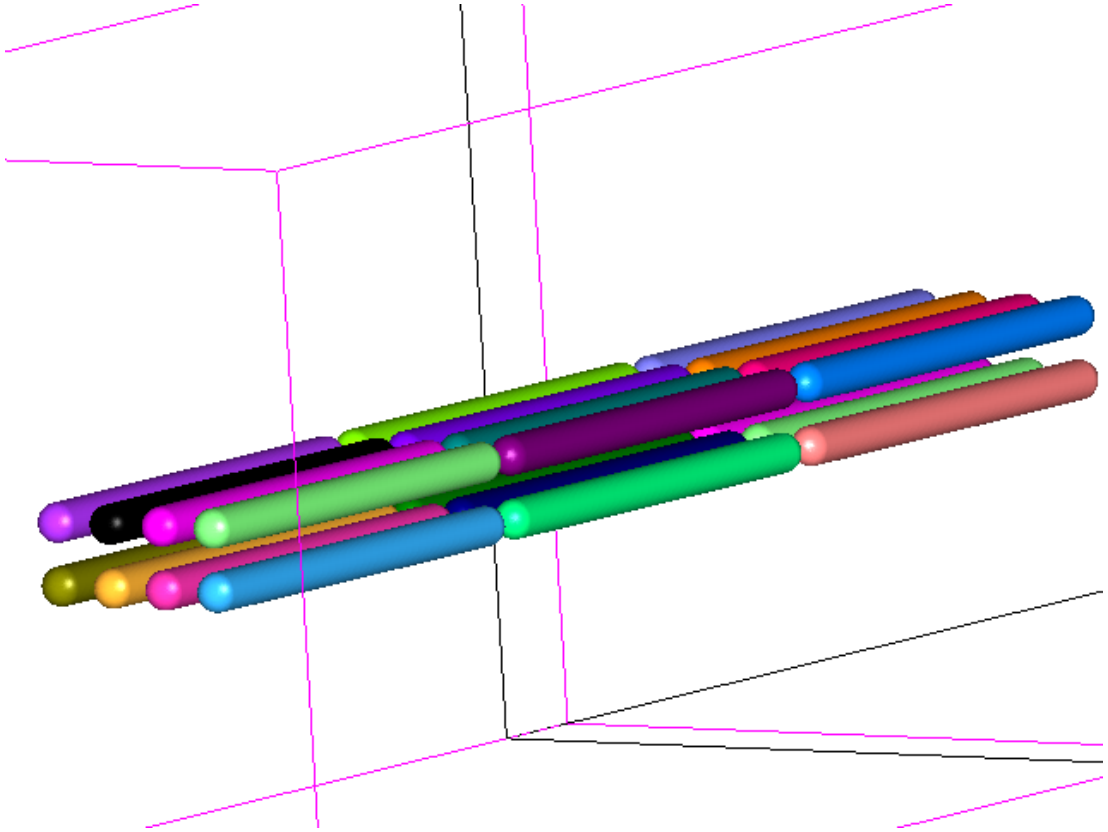
Dual NVIDIA GeForce RTX 3060

| | 1gpu | 2gpu/1gpu | 2gpuD/1gpu | tbb/1gpu | Suggar++/1gpu |
|---|---|---|---|---|---|
| CC | 0.208 | 5.0 | 2.1 | 13.6 | 55.0 |
| NC | 0.149 | 1.9 | 1.1 | 7.4 | 19.9 |

**CC: Cell Centered**
**NC: Node Centered**
**1gpu: single gpu (x16) wall clock time in seconds**
**2gpu: two gpus (x16, x4)**
**2gpuD: two gpus with duplication**
**tbb: threaded cpu execution only (16 core/32thr)**
**Suggar++ (16 core/32thr)**

# Darts4x2x3 Mixed Element Grids

- 29.8M nodes, 72.6M cells
  - 2 Background, 24 Dart grids





# Darts4x2x3 Mixed Element Grids: Wall Clock Comparison

Dual NVIDIA GeForce RTX 3060

| | 1gpu | 2gpu/1gpu | 2gpuD/1gpu | tbb/1gpu | Suggar++/1gpu |
|---|---|---|---|---|---|
| CC | 0.861 | 2.4 | 1.1 | 10.9 | 15.3 |
| NC | 0.396 | 2.3 | 1.0 | 11.2 | 12.2 |

NVIDIA A100-SXM4-80GB

| | 1gpu | 2gpu/1gpu | 2gpuD/1gpu | | Suggar++/1gpu |
|---|---|---|---|---|---|
| CC | 0.254 | 1.3 | 0.63 | | 52.0 |
| NC | 0.177 | 2.0 | 0.60 | | 27.3 |



# Darts4x2x3 Mixed Element Grids: Peer-to-Peer Transfer/Cell Centered

| | Data Transferred (GB) | Wall Clock Time (sec) | |
|---|---|---|---|
| | | A100-SXM4 | RTX 3060 |
| 2gpu | 9.52 | 0.342 | 2.075 |
| 2gpuD | 1.74 | 0.158 | 0.924 |



# CAGE Current Status & Future Work

- Ready for first release
- API for use with DiRTlib to communicate with flow solver
  - Tested with FUN3D for Wing Store test case

- Future work
  - Continue to improve performance
  - Add additional capabilities



# Summary

- Provided a brief discussion of GPUs and why Suggar++ is not suited to GPU execution
- Brief description of CAGE
- Brief discussion of typical overset assembly work and GPU execution
- Presented 3 testcases and showed performance of CAGE on GPUs
  - Significantly faster than Suggar++



# Questions?

Commercial distribution and support
for Suggar++ & CAGE provided by

**Celeritas Simulation Technology, LLC**

**http://www.CeleritasSimTech.com**

**Exportable under an EAR-99 license**